Pacgate AI - Two-AIPC Deployment Handbook

> Clone the repo on each machine, run the same install steps, and both machines become fully operational with deer-flow research and qm collaboration. Version 0.1.4 - 2026-09-04 Prerequisites: Docker Desktop, Ollama, Node.js 24+. `install.ps1` pulls the models listed in `ollama-models.txt`.

⚠ Significant findings (2026-09-02) — read before deploying AIPC #2

These were discovered during the AIPC #1 pilot and are already fixed in this repo. AIPC #2 must pull the updated code (from `pacgate-ai/pacgate-ai-pr`, see Stage 1) so it gets these fixes, not the older `JZKK720/pacgate-ai-pr` main.

1. deer-flow agent could not query pacgate's legal databases. Root cause: no tool was wired to pacgate-api's `/api/kb/search` (RAG) or `/api/search` (legal connectors), and the memory adapter silently fell back to local files (`PacgateMemoryStorage requires PACGATE_MATTER_ID`). Fix: a new `pacgate-mcp` service (FastMCP) exposes `pacgate_kb_search`, `pacgate_connector_search`, `pacgate_list_connectors` to deer-flow. Registered in `deer-flow-extensions-config.json` beside openviking.
2. openviking MCP had the wrong API key baked in. `deer-flow-extensions-config.json` used the app key (`OPENVIKING_API_KEY`) but openviking's `root_api_key` is `OPENVIKING_ROOT_API_KEY`. A wrong key made openviking return 401, which rolled back the entire MCP tool load (deer-flow uses `asyncio.gather`), so no MCP tools appeared. Fix: use `OPENVIKING_ROOT_API_KEY` (template now uses `${OPENVIKING_ROOT_API_KEY}`).
3. `docker compose up -d --force-recreate deer-flow` wipes deer-flow's local DB. The SQLite DB, admin user, threads, and `.jwt_secret` live at `/app/backend/.deer-flow/` inside the container (not mounted). Recreating the container loses them → the frontend gets 401 and the `/setup` page appears. Use `docker compose restart deer-flow` for config changes; only recreate if you accept losing the local DB (then re-run `/setup`).
4. QM sign-in needs a `RESEND_API_KEY`, not Outlook SMTP. The old SMTP path (`smtp.office365.com` + app password) is broken — Microsoft retired Basic Auth / app passwords for Exchange Online (Sep 2025). `qm check` failed with `535 5.7.139 Authentication unsuccessful`. Fix: qm's auth broker now uses the Resend transport (`AUTH_EMAIL_TRANSPORT=resend`). You must supply a `RESEND_API_KEY` in `deploy/qm-pacgate/.env` (see Stage 4).
5. The qm web-ui cannot self-authenticate. Its server (`/app/server/index.ts`) sets `AUTH_MODE = COOKIE_AUTH ? "dev" : "portal"`. Because `CORE_SIGNING_SECRET` is set, it's in portal mode and requires a portal-issued identity token. There is no no-secret way to reach the web-ui directly — you must run `portal+auth` (Resend) or an external OIDC provider. `ADMIN_GRANTS` is an authorization seed, not a sign-in.
6. Git push to `JZKK720/pacgate-ai-pr` is blocked for the `pacgate-ai` account (403, needs 2FA grant). Workaround: the `pacgate-ai` account can create a fork and push there. The fork `pacgate-ai/pacgate-ai-pr` now carries all fixes on `main`.

Architecture: two identical machines

Both AIPCs run the complete stack:

```
Each AIPC machine:
  nginx :8089  -> pacgate-api :8080 (Rust metadata API)
                -> deer-flow  :8001 (research workspace)
  Postgres :5432 (local metadata DB)
  OpenViking :1933 (long-term memory lane, MCP)
  qm :8182 (co-working workspace, runs via `qm up`)
  Ollama :11434 (native, GPU/NPU)
```

Each machine is self-contained and independently operational. Lawyers on either machine can use both research mode (deer-flow at `http://localhost:8089/research/`) and collaboration mode (qm at `http://localhost:8182`) without depending on the other machine.

If you later want shared matter data across both machines, connect them with a private mesh (Tailscale or WireGuard) and decide on a sync or single-authority model. That is a post-pilot decision, not a deployment prerequisite.

What you need before starting

- GitHub access to `JZKK720/pacgate-ai-pr` (private repo) — a PAT or `gh auth login`
- Docker Desktop running on both AIPCs
- Ollama running on both AIPCs (`install.ps1` pulls the models it needs)
- `ollama signin` completed on each AIPC if the cloud-tagged deepseek models are in use
- Node.js 24+ installed on both AIPCs (for qm)
- No `docker login ghcr.io` needed — the Pacgate runtime images are published as public GHCR packages (see Stage 0). Only the source repo is private.

Stage 0: Runtime images (dev machine, already done)

The runtime is published on GHCR and needs no rebuild on the AIPC:

Image	Status
ghcr.io/jzkk720/pacgate-api:0.1.2	Published. Fixes the 0.1.1 container-networking bug (LLM router honors OLLAMA_BASE_URL, per-tenant model overrides applied).
ghcr.io/jzkk720/deer-flow-pacgate:0.1.0	Published. Thin wrapper on the upstream deer-flow backend; unchanged.
ghcr.io/volcengine/openviking@sha256:46f9e34c⋯	Pinned by digest in compose.prod.yaml. Upstream public image.

Both Pacgate packages must be set to public visibility on GHCR so an AIPC can pull without registry credentials. Verify before rollout:

```
# Expect HTTP 200 with no docker login. 401/403 means the package is still private.
# (The Accept header is required — omit it and a public manifest returns 404, not 200.)
$acc = "application/vnd.oci.image.index.v1+json,application/vnd.docker.distribution.manifest.list.v2+json,ap
$t = (Invoke-RestMethod "https://ghcr.io/token?scope=repository:jzkk720/pacgate-api:pull").token
(Invoke-WebRequest "https://ghcr.io/v2/jzkk720/pacgate-api/manifests/0.1.2" -Headers @{Authorization="Bearer
```

To flip it (GitHub web UI — the API route 404s for personal accounts): GitHub → your profile → Packages → `pacgate-api` → Package settings → Visibility → Public → Save. Repeat for `deer-flow-pacgate`. This is safe: the images contain only the compiled binary and SQL migrations, every secret is injected at runtime via `.env`, and the installer already has full source access to the same code.

Only rebuild and push if the Rust source changes, from the dev machine:

```
cd c:\Users\cubecloud-io\github-pr\pacgate-ai-pr
docker build -t ghcr.io/jzkk720/pacgate-api:0.1.3 -f pacgate-ai/Dockerfile ./pacgate-ai
docker push ghcr.io/jzkk720/pacgate-api:0.1.3
```

Then bump the tag in `deploy/client-bundle/compose.prod.yaml`.

Do not rebuild on the AIPC — the pilot runs the published digests.

> Port note: the stack binds nginx to host port `8089` (the value committed in `deploy/client-bundle/compose.prod.yaml`). If that port is already in use on the machine, edit the `ports:` entry for `nginx` and use the new port in all verification URLs below.

Stage 1: Clone the repo on each AIPC

On both machines:

```
cd C:\
git clone https://github.com/pacgate-ai/pacgate-ai-pr.git
cd pacgate-ai-pr
```

> AIPC #2 note: clone from the `pacgate-ai/pacgate-ai-pr` fork (it carries all the 2026-09-02 fixes on `main`). The `pacgate-ai` account owns it, so it's writable and always up to date. If you must use `JZKK720/pacgate-ai-pr`, pull the `feat/deer-flow-pacgate-mcp` branch (or apply the patches in `patches/`) to get the same fixes.

If the repo is private and GitHub prompts for credentials, use a personal access token or the GitHub CLI (`gh auth login`).

Stage 2: Deploy the core stack (both machines, identical steps)

Run these steps on each AIPC. The Docker Compose stack starts pacgate-api, Postgres, nginx, and deer-flow.

```
cd C:\pacgate-ai-pr\deploy\client-bundle
copy .env.example .env
notepad .env
```

Fill in these values:

```
PACGATE_DB_PASSWORD=<generate a strong password>
PACGATE_JWT_SECRET=<generate a random hex string>
PACGATE_TENANT_ID=pacgate-law
OPENVIKING_ROOT_API_KEY=<generate a 32-char hex string>
OPENVIKING_API_KEY=<generate a 32-char hex string>
```

`OPENVIKING_API_KEY` is required — the installer renders `deer-flow-extensions-config.json` from it and stops with an error if it is missing or left as `change-me`.

Generate secrets if you need them:

```
# DB password (16 hex)
-join ((1..16) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# JWT secret (32 hex)
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })

# OpenViking keys (32 hex each) — generate a fresh value per line
-join ((1..32) | ForEach-Object { '{0:x}' -f (Get-Random -Maximum 16) })
```

Run the installer:

```
.\install.ps1
```

The installer pulls the (public, no-login) GHCR images, renders `OPENVIKING_CONF_CONTENT` and `deer-flow-extensions-config.json` from the `.env` secrets, starts the Docker Compose stack, and pulls the Ollama models listed in `ollama-models.txt`. If models are already pulled, this step is fast.

Verify the core stack:

```
docker compose -f compose.prod.yaml ps
curl http://localhost:8089/health
```

Expected: all five containers running (pacgate-db, pacgate-api, deer-flow, openviking, nginx) and `/health` returns `ok`.

Stage 3: Seed the tenant and register users (both machines)

On each machine, seed the default tenant and register the admin user:

```
# Seed the tenant
docker exec pacgate-db psql -U pacgate -c "INSERT INTO tenants (name, slug) VALUES ('Pacgate Law', 'pacgate-

# Register the admin user
$body = @{email="admin@pacgate-law.com"; password="<strong-password>"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/api/auth/register" -Method POST -Body $body -ContentType "appl
```

Register a qm bridge service account (needed by qm to authenticate with pacgate-api):

```
$body = @{email="qm-bridge@pacgate.local"; password="<strong-bridge-password>"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/api/auth/register" -Method POST -Body $body -ContentType "appl
```

Stage 3.5: Verify OpenViking memory service (both machines)

OpenViking is the long-term memory lane: deer-flow and qm store conversational context there and recall it in later sessions. It starts as part of the compose stack.

```
curl http://localhost:1933/health
```

Expected: `{"status":"ok","healthy":true,...}`. The installer renders the OpenViking config (Ollama embedding + VLM) into `.env` as `OPENVIKING_CONF_CONTENT` and seeds the server's `ov.conf` on first boot.

Functional check (optional, uses the root key from `.env`):

```
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: application/json" -H "A
```

Expected: a tool list including `find`, `search`, `read`, `remember`.

Boundary rule: OpenViking stores conversational context only (decisions, preferences, working knowledge). Matter documents and T1-T4-controlled content stay in pacgate-api/pacgate-rag.

Stage 4: Bootstrap qm (both machines, identical steps)

qm runs separately from the Docker Compose stack. Bootstrap it on each machine after the core stack is healthy.

```
cd C:\pacgate-ai-pr\deploy\client-bundle
.\setup-qm.ps1
```

The script prompts for: - Administrator work email (lowercased) - Pacgate bridge email: `qm-bridge@pacgate.local` - Pacgate bridge password: the one you registered in Stage 3

The script generates signing secrets, creates `.env` in the qm-pacgate directory, validates the config with `qm check`, and builds the sandbox image with `qm sandbox build`.

QM sign-in requires a Resend API key. The qm auth broker delivers sign-in magic links via Resend (`AUTH_EMAIL_TRANSPORT=resend`), not Outlook SMTP (Microsoft retired Basic Auth / app passwords for Exchange Online). Before `qm up` will start `portal+auth`, set `RESEND_API_KEY` in `deploy/qm-pacgate/.env`:

```
RESEND_API_KEY=re_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

Also set `AUTH_EMAIL_FROM` to a Resend-verified sender (an Outlook address is not verified). Either verify a real domain (e.g. `pacgate-law.com`) in Resend, or use Resend's test sender for now:

```
AUTH_EMAIL_FROM="PacGate <onboarding@resend.dev>"
```

Start qm:

```
cd C:\pacgate-ai-pr\deploy\qm-pacgate
node_modules\.bin\qm.cmd up
```

> Note: `npm exec qm -- up` may be blocked by the PowerShell execution policy (`npm.ps1`). Use `node_modules\.bin\qm.cmd up` instead.

Verify qm:

```
# Open http://localhost:8182  (web-ui) — requires a portal identity token
# Open http://localhost:8181  (portal) — the sign-in front door
# Sign in with the admin email (magic link via Resend)
# Send a test message
# Ask: "List available pacgate workflows"
```

> Web-ui auth reality: the qm web-ui (`:8182`) cannot authenticate anyone on its own — it must be reached through the portal (`:8181`), which issues the identity token. If you open `:8182` directly you'll see "reached through the portal." Always go through `:8181`.

> Dev-mode alternative (pilot / single-user): for a local pilot you can run qm in dev/cookie mode instead of the portal. Recreate `qm-pacgate-core` with `NODE_ENV=development` + `ALLOW_UNAUTHENTICATED_CORE=1` and no `CORE_SIGNING_SECRET`, and `qm-pacgate-web-ui` with no `CORE_SIGNING_SECRET`. Then `POST /signin` works directly at `:8182` with `{"user":"<principal>"}` and no Resend key is needed. This is not production-correct (no auth) — use it only for a single-user pilot. See `deer-flow/docs/pacgate/QM-WEBUI-8182-SIGNIN-FIX-PLAN.md` for the exact commands.

Stage 5: Verify deer-flow (both machines)

On each machine, verify the research workspace:

```
# Open http://localhost:8089/research/
# Select or create a matter
# Ask: "Summarize recent force majeure case law in China"
# Verify: response includes citations
# Verify: response is saved to matter memory
```

Stage 5.5: Full-loop operations — deer-flow ↔ QM ↔ OpenViking

Both workspaces share one OpenViking long-term memory lane and one pacgate-api metadata store. This section documents how the two systems talk to each other and how to verify the loop end-to-end.

Topology (verified 2026-09-04)

```
pacgate-ai-bundle_default  (Docker Compose network)
■■■ openviking :1933   ← long-term memory (MCP: find/search/read/remember)
■■■ pacgate-api :8080  ← metadata API (matters/workflows/connectors)
■■■ pacgate-mcp :8000  ← FastMCP bridge exposing pacgate KB/connector search
■■■ deer-flow :8001    ← research workspace (consumes openviking + pacgate-mcp)
■■■ nginx :8089        ← ingress (deer-flow frontend + /pacgate/ API)

qm-pacgate  (qm up network)
■■■ qm-pacgate-core    ← co-working agent runtime
■■■ qm-pacgate-web-ui  ← browser chat UI (:8182)
■■■ qm-pacgate-pg      ← qm Postgres

Bridge: qm-pacgate-core is ALSO joined to pacgate-ai-bundle_default, so it can
reach openviking:1933, pacgate-api:8080, and host.docker.internal:11434 (Ollama).
```

How the loop works

1. deer-flow → OpenViking: `deer-flow-extensions-config.json` registers `openviking` as an HTTP MCP server at `http://openviking:1933/mcp` with the root API key (`OPENVIKING_ROOT_API_KEY`). Research runs store and recall conversational context there.
2. deer-flow → pacgate: `pacgate-mcp` (FastMCP) exposes `pacgate_kb_search` / `pacgate_connector_search` / `pacgate_list_connectors` to deer-flow, backed by pacgate-api's RAG + legal connectors.
3. QM → OpenViking: the QM core has `OPENVIKING_URL=http://openviking:1933` and `OPENVIKING_API_KEY` (the root key). The `pacgate-qm` sandbox tool (`ov-remember` / `ov-search` / `ov-read`) calls OpenViking from inside the agent sandbox at `http://host.docker.internal:1933` with `OPENVIKING_ACCOUNT=pacgate-law` + `OPENVIKING_USER`.
4. QM → pacgate: the `pacgate-qm` sandbox tool logs into pacgate-api (`PACGATE_API_EMAIL` / `PACGATE_API_PASSWORD`) to discover workflows, bind a QM scope to a matter, read/write matter memory, and execute workflows.

Verify the full loop

```
# 1. OpenViking MCP responds (root key from .env)
$key = (Get-Content .env | Select-String '^OPENVIKING_ROOT_API_KEY=').Line.Split('=')[1]
$body = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
curl.exe -s -X POST http://localhost:1933/mcp -H "X-API-Key: $key" -H "Content-Type: application/json" -H "A
# Expected: tools find/search/read/remember

# 2. QM core reaches openviking + pacgate-api + Ollama
docker exec qm-pacgate-core sh -c "curl -s -o /dev/null -w 'openviking:%{http_code}\n' http://openviking:193

# 3. QM sign-in works (dev/cookie mode)
$body = '{"user":"admin@pacgate-law.com"}'
curl.exe -s -X POST http://localhost:8182/signin -H "Content-Type: application/json" -d $body
# Expected: {"ok":true,"user":"admin@pacgate-law.com"}
```

QM ↔ bundle network join (idempotent)

QM core is joined to the bundle network so it can resolve `openviking` and `pacgate-api` by name. If a `qm up`/`qm down` cycle drops the join, re-apply it:

```
docker network connect pacgate-ai-bundle_default qm-pacgate-core
```

> Note: QM is intentionally not a Docker Compose service in the bundle. It is managed by the `@yc-software/qm` CLI (`qm up` / `qm down`) with its own lifecycle. The network join is the only coupling — keep it that way. Do not rewrite QM as compose services; that fights the qm CLI and would recreate the containers on every `qm up`.

Stage 6: Smoke test checklist (both machines)

Run this checklist on each AIPC independently.

Core stack

- [ ] `docker compose -f compose.prod.yaml ps` shows 5 services up (incl. openviking)
- [ ] `curl http://localhost:8089/health` returns `ok`
- [ ] `curl http://localhost:1933/health` returns healthy JSON
- [ ] Postgres has the `pacgate-law` tenant
- [ ] Admin user can log in at `http://localhost:8089/api/auth/login`
- [ ] deer-flow returns a real research response at `http://localhost:8089/research/`

qm collaboration

- [ ] `npm exec qm -- status` shows qm running
- [ ] `http://localhost:8182` loads the qm web UI
- [ ] Admin can sign in
- [ ] qm can list Pacgate workflow categories
- [ ] qm can execute one Pacgate workflow through the bridge

Ollama

- [ ] `ollama list` shows the required models
- [ ] deer-flow can call Ollama for inference
- [ ] qm can call Ollama for inference

Data

- [ ] `./data/tenants/` directory exists and is writable
- [ ] `./openviking/` directory exists and persists across restarts
- [ ] Document upload works through the API
- [ ] Matter memory persists after a deer-flow research run
- [ ] Cross-session recall: a fact stored via OpenViking `remember` is recalled via `search` in a later session

Managing the stack after deployment

Start and stop

```
# Start core stack
docker compose -f compose.prod.yaml up -d

# Stop core stack
docker compose -f compose.prod.yaml down

# Start qm
cd C:\pacgate-ai-pr\deploy\qm-pacgate
npm exec qm -- up

# Stop qm
npm exec qm -- down
```

Update to a new version

```
cd C:\pacgate-ai-pr
git pull
cd deploy\client-bundle
.\install.ps1 -Update
```

The update pulls new GHCR images and restarts containers. Data is preserved: - `./data/tenants/` (volume mount) - matters, documents, memory - Postgres data (named volume) - metadata database

Switch models

deer-flow (research workspace): 1. Edit `deer-flow-config.yaml` - reorder the `models` list (first entry = default) 2. Restart: `docker compose -f compose.prod.yaml restart deer-flow`

qm (co-working workspace): 1. Edit `qm-pacgate/qm.config.jsonc` - change `MODEL_NAME` 2. Restart: `cd qm-pacgate && npm exec qm -- down && npm exec qm -- up`

Register new users

```
$body = @{email="<user>@pacgate-law.com"; password="<password>"} | ConvertTo-Json
Invoke-RestMethod -Uri "http://localhost:8089/api/auth/register" -Method POST -Body $body -ContentType "appl
```

Backup the database

```
docker exec pacgate-db pg_dump -U pacgate pacgate > backup.sql
```

Check logs

```
docker compose -f compose.prod.yaml logs -f pacgate-api
docker compose -f compose.prod.yaml logs -f deer-flow
```

Known limitations

- QM local-model routing is non-durable. To make QM chat work against local Ollama, a custom model entry (`glm-5.3-flash:cloud`) was added to the QM core's `src/model/pi-models.ts` (in the container's writable layer). This edit is lost whenever the core container is recreated (e.g. `qm up` after `qm down`, or a manual `docker rm -f qm-pacgate-core`). After any recreate, `glm-5.3-flash:cloud` disappears from `GET /v1/surface-config` → `webuiModels`, and chat turns return 403 "that model isn't available". To re-apply: `bash # 1. Add the custom entry to MODEL_REGISTRY in /app/src/model/pi-models.ts # { id: "glm-5.3-flash:cloud", name: "GLM 5.3 Flash (Ollama)", fastMode: false, # webui: true, base: true, # custom: { template: "gpt-4.1-mini", baseUrl: "http://host.docker.internal:11434/v1" } } # 2. Extend ModelEntry with optional custom:{template,baseUrl} and handle it in resolveModel() # 3. Ensure OPENAI_API_KEY=ollama-local is set on the core (Ollama ignores the value) # 4. Restart the core` For a durable fix, commit the change to the qm source repo and rebuild the image, or stand up a proxy (e.g. LiteLLM) that maps the openai provider to Ollama.
- Each machine has its own independent Postgres and `./data/tenants/` directory. Matter data is not shared between machines unless you later add a private mesh and a sync or single-authority model.
- The PkuLaw connector token is expired. Regenerate it at `https://mcp.pkulaw.com` and set `PKULAW_API_KEY` in `.env` if China-law search is needed during the pilot.
- Four WASM crates (citation-check, clause-parser, doc-validator, rule-engine) remain stubs. These are future-blueprint work and do not affect Phase 1 pilot functionality.
- Model selection: the API defaults to models that may not exist on the target machine. After Stage 3, apply per-tenant model overrides so the LLM tiers point at models actually present in `ollama list` on that machine. Recommended pilot set (benchmarked 2026-08-28): `gemma4:12b-it-qat` (Main — 13s/tool-round, schema-valid tool calls, verified end-to-end), `qwen3.8:27b-mtp-q4_K_M` (Mid — 73s/tool-round, stronger quality for batch tabular review), `nomic-embed-text:latest` (embeddings). Avoid reasoning-mode models (e.g. nemotron) for interactive tiers — they can hang long docx generations. See `plans/007-aipc-full-installation-handoff.md` Appendix A for the SQL template.

Files referenced

File	Purpose
deploy/client-bundle/compose.prod.yaml	Docker Compose for pacgate-api + deer-flow + Postgres + nginx
deploy/client-bundle/install.ps1	One-click Windows installer for the core stack

File	Purpose
deploy/client-bundle/setup-qm.ps1	qm bootstrap script (secrets, config, sandbox build)
deploy/client-bundle/.env.example	Template for client secrets
deploy/client-bundle/ollama-models.txt	Models to pre-pull
deploy/client-bundle/deer-flow-config.yaml	Multi-model deer-flow config (5 models, switchable)
deploy/qm-pacgate/qm.config.jsonc	qm local deployment config
deploy/SETUP-AND-OPERATIONS.md	Full 3-day on-site install guide (reference)
deploy/DEPLOYMENT-GUIDE.md	Engineer-level deployment details (reference)
deer-flow/docs/pacgate/QM-WEBUI-8182-SIGNIN-FIX-PLAN.md	QM sign-in + model-routing fix plan (diagnosis + exact commands)
deer-flow/docs/pacgate/QM-WEBUI-8182-AUTH-DIAGNOSIS.md	QM portal-auth bottleneck diagnosis